

2017 special issue

Sigmoid-weighted linear units for neural network function approximation in reinforcement learning

Stefan Elfving^{a,*}, Eiji Uchibe^{a,b}, Kenji Doya^b^a Department of Brain Robot Interface, ATR Computational Neuroscience Laboratories, 2-2-2 Hikaridai, Seikacho, Soraku-gun, Kyoto 619-0288, Japan^b Okinawa Institute of Science and Technology Graduate University, 1919-1 Tancha, Onna-son, Okinawa 904-0495, Japan

ARTICLE INFO

Article history:

Available online 11 January 2018

Keywords:

Reinforcement learning
Sigmoid-weighted linear unit
Function approximation
Tetris
Atari 2600
Deep learning

ABSTRACT

In recent years, neural networks have enjoyed a renaissance as function approximators in reinforcement learning. Two decades after Tesauro's TD-Gammon achieved near top-level human performance in backgammon, the deep reinforcement learning algorithm DQN achieved human-level performance in many Atari 2600 games. The purpose of this study is twofold. First, we propose two activation functions for neural network function approximation in reinforcement learning: the sigmoid-weighted linear unit (SiLU) and its derivative function (dSiLU). The activation of the SiLU is computed by the sigmoid function multiplied by its input. Second, we suggest that the more traditional approach of using on-policy learning with eligibility traces, instead of experience replay, and softmax action selection can be competitive with DQN, without the need for a separate target network. We validate our proposed approach by, first, achieving new state-of-the-art results in both stochastic SZ-Tetris and Tetris with a small 10×10 board, using TD(λ) learning and shallow dSiLU network agents, and, then, by outperforming DQN in the Atari 2600 domain by using a deep Sarsa(λ) agent with SiLU and dSiLU hidden units.

© 2017 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

Neural networks have enjoyed a renaissance as function approximators in reinforcement learning (Sutton & Barto, 1998) in recent years. The DQN algorithm (Mnih et al., 2015), which combines Q-learning with a deep neural network, experience replay, and a separate target network, achieved human-level performance in many Atari 2600 games. Since the development of the DQN algorithm, there have been several proposed improvements, both to DQN specifically and deep reinforcement learning in general. van Hasselt, Guez, and Silver (2015) proposed double DQN to reduce overestimation of the action values in DQN and Schaul, Quan, Antonoglou, and Silver (2016) developed a framework for more efficient replay by prioritizing experiences of more important state transitions. Wang et al. (2016) proposed the dueling network architecture for more efficient learning of the action value function by separately estimating the state value function and the advantages of each action. Mnih et al. (2016) proposed a framework for asynchronous learning by multiple agents in parallel, both for value-based and actor-critic methods. To date, the most impressive application of using deep reinforcement learning is AlphaGo (Silver

et al., 2016, 2017), which has achieved superhuman performance in the ancient board game Go.

The purpose of this study is twofold. First, motivated by the high performance of the expected energy restricted Boltzmann machine (EE-RBM) in our earlier studies (Elfving, Uchibe, & Doya, 2015, 2016), we propose two activation functions for neural network function approximation in reinforcement learning: the sigmoid-weighted linear unit (SiLU) and its derivative function (dSiLU). The activation of the SiLU is computed by the sigmoid function multiplied by its input. After we first proposed the SiLU (Elfving, Uchibe, & Doya, 2017), Ramachandran, Zoph, and Le (2017) recently performed a comprehensive comparison between the SiLU, the rectifier linear unit (ReLU; Hahnloser, Sarpeshka, Mahowald, Douglas, & Seung, 2000), and 6 other activation functions in the supervised learning domain. They found that the SiLU consistently outperformed the other activation functions when tested in 3 deep architectures on CIFAR-10/100 (Krizhevsky, 2009), in 5 deep architectures on ImageNet (Deng et al., 2009), and on 4 test sets for English-to-German machine translation.

Second, we suggest that the more traditional approach of using on-policy learning with eligibility traces, instead of experience replay, and softmax action selection with simple annealing can be competitive with DQN, without the need for a separate target network. Our approach is something of a throwback to the approach used by Tesauro (1994) to develop TD-Gammon more than two decades ago. Using a neural network function approximator and

* Corresponding author.

E-mail addresses: elfwing@atr.jp (S. Elfving), uchibe@atr.jp (E. Uchibe), doya@oist.jp (K. Doya).

TD(λ) learning (Sutton, 1988), TD-Gammon reached near top-level human performance in backgammon, which to this day remains one of the most impressive application of reinforcement learning.

To evaluate our proposed approach, we first test the performance of shallow network agents with SiLU, ReLU, dSiLU, and sigmoid hidden units in stochastic SZ-Tetris, which is a simplified but difficult version of Tetris. The best agent, the dSiLU network agent, improves the average state-of-the-art score by 20%. In stochastic SZ-Tetris, we also train deep network agents using raw board configurations as states. An agent with SiLUs in the convolutional layers and dSiLUs in the fully-connected layer (SiLU–dSiLU) outperforms the previous state-of-the-art average final score. We thereafter train a dSiLU network agent in standard Tetris with a smaller, 10×10 , board size, achieving a state-of-the-art score in this more competitive version of Tetris as well. We then test a deep SiLU–dSiLU network agent in the Atari 2600 domain. It improves the mean DQN normalized scores achieved by DQN and double DQN by 232% and 161%, respectively, in 12 unbiasedly selected games. We finally analyze the ability of on-policy value-based reinforcement learning to accurately estimate the expected discounted returns and the importance of softmax action selection for the games where our proposed agents performed particularly well.

2. Method

2.1. TD(λ) and Sarsa(λ)

In this study, we use two reinforcement learning algorithms: TD(λ) (Sutton, 1988) and Sarsa(λ) (Rummery & Niranjan, 1994; Sutton, 1996). TD(λ) learns an estimate of the state-value function, V^π , and Sarsa(λ) learns an estimate of the action-value function, Q^π , while the agent follows policy π . If the approximated value functions, $V_t \approx V^\pi$ and $Q_t \approx Q^\pi$, are parameterized by the parameter vector θ_t , then the gradient-descent learning update of the parameters is computed by

$$\theta_{t+1} = \theta_t + \alpha \delta_t \mathbf{e}_t, \quad (1)$$

where the TD-error, δ_t , is

$$\delta_t = r_t + \gamma V_t(s_{t+1}) - V_t(s_t) \quad (2)$$

for TD(λ) and

$$\delta_t = r_t + \gamma Q_t(s_{t+1}, a_{t+1}) - Q_t(s_t, a_t) \quad (3)$$

for Sarsa(λ). The eligibility trace vector, $\mathbf{e}_t$, is

$$\mathbf{e}_t = \gamma \lambda \mathbf{e}_{t-1} + \nabla_{\theta_t} V_t(s_t), \quad \mathbf{e}_0 = \mathbf{0}, \quad (4)$$

for TD(λ) and

$$\mathbf{e}_t = \gamma \lambda \mathbf{e}_{t-1} + \nabla_{\theta_t} Q_t(s_t, a_t), \quad \mathbf{e}_0 = \mathbf{0}, \quad (5)$$

for Sarsa(λ). Here, s_t is the state at time t , a_t is the action selected at time t , r_t is the reward for taking action a_t in state s_t , α is the learning rate, γ is the discount factor of future rewards, λ is the trace-decay rate, and $\nabla_{\theta_t} V_t$ and $\nabla_{\theta_t} Q_t$ are the vectors of partial derivatives of the function approximators with respect to each component of θ_t .

2.2. Sigmoid-weighted linear units

We proposed the EE-RBM as a function approximator in reinforcement learning (Elfving et al., 2016). In the case of state-value based learning, given a state vector $\mathbf{s}$, an EE-RBM approximates the state-value function V by the negative expected energy of an

RBM (Freund & Haussler, 1992; Hinton, 2002; Smolensky, 1986) network:

$$V(\mathbf{s}) = \sum_k z_k \sigma(z_k) + \sum_i b_i s_i, \quad (6)$$

$$z_k = \sum_i w_{ik} s_i + b_k, \quad (7)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}. \quad (8)$$

Here, z_k is the input to hidden unit k , $\sigma(\cdot)$ is the sigmoid function, b_i is the bias weight for input unit s_i , w_{ik} is the weight connecting state s_i and hidden unit k , and b_k is the bias weight for hidden unit k . Note that Eq. (6) can be regarded as the output of a one-hidden layer feedforward neural network with hidden unit activations computed by $z_k \sigma(z_k)$ and with uniform output weights of one. In this study, motivated by the high performance of the EE-RBM in both the classification (Elfving et al., 2015) and the reinforcement learning (Elfving et al., 2016) domains, we propose the SiLU as an activation function for neural network function approximation in reinforcement learning. The activation a_k of the k th SiLU for input z_k is computed by the sigmoid function multiplied by its input (i.e., equal to the contribution from a hidden node to the value function in an EE-RBM):

$$a_k(z_k) = z_k \sigma(z_k). \quad (9)$$

For z_k -values of large magnitude, the activation of the SiLU is approximately equal to the activation of the ReLU (see left panel in Fig. 1), i.e., the activation is approximately equal to zero for large negative z_k -values and approximately equal to z_k for large positive z_k -values. Unlike the ReLU (and other commonly used activation units such as sigmoid and tanh units), the activation of the SiLU is not monotonically increasing. Instead, it has a global minimum value of approximately -0.28 for $z_k \approx -1.28$. An attractive feature of the SiLU is that it has a self-stabilizing property, which we demonstrated experimentally in Elfving et al. (2015). The global minimum, where the derivative is zero, functions as a “soft floor” on the weights that serves as an implicit regularizer that inhibits the learning of weights of large magnitudes.

In Elfving et al. (2015), we discovered that the derivative function of the SiLU (i.e., the derivative of the contribution from a hidden node to the output in an EE-RBM) looks like a steeper and “overshooting” version of the sigmoid function. In this study, we call this function the dSiLU and we propose it as a competitive alternative to the sigmoid function in neural network function approximation in reinforcement learning. The activation of the dSiLU is computed by the derivative of the SiLU (see right panel in Fig. 1):

$$a_k(z_k) = \sigma(z_k) (1 + z_k(1 - \sigma(z_k))). \quad (10)$$

The dSiLU has a maximum value of approximately 1.1 and a minimum value of approximately -0.1 for $z_k \approx \pm 2.4$, i.e., the solutions to the equation $z_k = -\log((z_k - 2)/(z_k + 2))$.

2.3. Action selection

We use softmax action selection with a Boltzmann distribution in all experiments. For Sarsa(λ), the probability to select action a in state s is defined as

$$\pi(a|s) = \frac{\exp(Q(s, a)/\tau)}{\sum_b \exp(Q(s, b)/\tau)}. \quad (11)$$

For the model-based TD(λ) algorithm, we select an action a in state s that leads to the next state s' with a probability defined as

$$\pi(a|s) = \frac{\exp(V(f(s, a))/\tau)}{\sum_b \exp(V(f(s, b))/\tau)}. \quad (12)$$

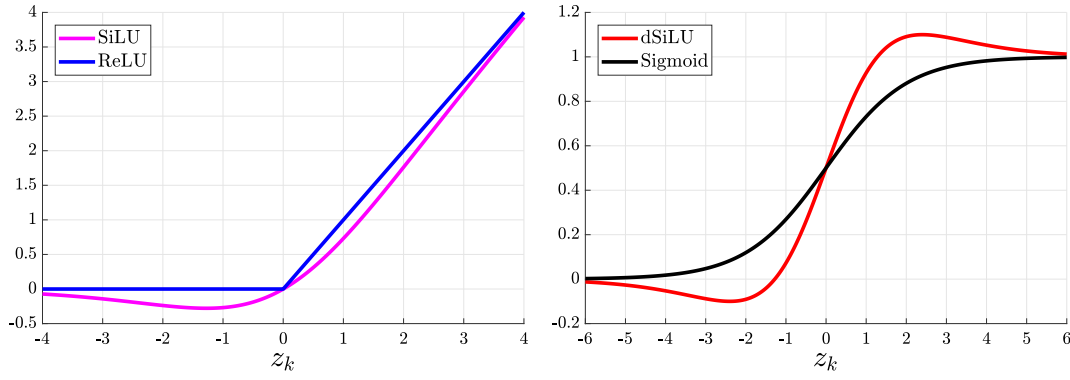


Fig. 1. The activation functions of the SiLU and the ReLU (left panel), and the dSiLU and the sigmoid unit (right panel).

Here, $f(s, a)$ returns the next state s' according to the deterministic state transition dynamics and τ is the temperature that controls the trade-off between exploration and exploitation. We used hyperbolic annealing of the temperature, and the temperature was decreased after every episode i :

$$\tau(i) = \frac{\tau_0}{1 + \tau_k i}. \quad (13)$$

Here, τ_0 is the initial temperature and τ_k controls the rate of annealing.

3. Experiments

3.1. SZ-Tetris

Szita and Szepesvári (2010) proposed stochastic SZ-Tetris (Burgiel, 1997) as a benchmark for reinforcement learning that preserves the core challenges of standard Tetris but allows faster evaluation of different strategies due to shorter episodes by removing easier tetrominos. Stochastic SZ-Tetris is played on a board of standard Tetris size with a width of 10 and a height of 20. In each time step, either an S-shaped tetromino or a Z-shaped tetromino appears with equal probability. The agent selects a rotation (lying or standing) and a horizontal position within the board. In total, there are 17 possible actions for each tetromino (9 standing and 8 lying horizontal positions). After the action selection, the tetromino drops down the board, stopping when it hits another tetromino or the bottom of the board. If a row is completed, then it disappears. The agent gets a score of +1 point for each completed row. An episode ends when a tetromino does not fit within the board.

The standard learning approach for Tetris has been to use a model-based setting and define the evaluation function or state-value function as the linear combination of hand-coded features. Value-based reinforcement learning algorithms have a lousy track record using this approach. In regular Tetris, their reported performance levels are many magnitudes lower than black-box methods such as the cross-entropy (CE) method and evolutionary approaches. In stochastic SZ-Tetris, the reported scores for a wide variety of reinforcement learning algorithms are either approximately zero (Szita & Szepesvári, 2010) or in the single digits.¹

Value-based reinforcement learning has had better success in stochastic SZ-Tetris when using non-linear neural network based function approximators. Faußer and Schwenker (2013) achieved a score of about 130 points using a shallow neural network function approximator with sigmoid hidden units. They improved the result to about 150 points by using an ensemble approach consisting of ten neural networks. We achieved an average score of

about 200 points using three different neural network function approximators: an EE-RBM, a free energy RBM, and a standard neural network with sigmoid hidden units (Elfving et al., 2016). Jaskowski, Szubert, Liskowski, and Krawiec (2015) achieved the current state-of-the-art results using systematic n-tuple networks as function approximators: average scores of 220 and 218 points achieved by the evolutionary VD-CMA-ES method and TD-learning, respectively, and the best mean score in a single run of 295 points achieved by TD-learning.

In this study, we use the TD(λ) algorithm and softmax action selection to compare the performance of different hidden activation units in two learning settings: (1) shallow network agents with one hidden layer using hand-coded state features and (2) deep network agents using raw board configurations as states, i.e., a state node is set to one if the corresponding board cell was occupied by a tetromino and set to zero otherwise.

In the setting with state features, we trained shallow network agents with SiLU, ReLU, dSiLU, and sigmoid hidden units. We used the same experimental setup as used in our earlier work (Elfving et al., 2016). The networks consisted of one hidden layer with 50 hidden units and a linear output layer. The features were similar to the original 21 features proposed by Bertsekas and Ioffe (1996), except for not including the maximum column height and using the differences in column heights instead of the absolute differences. The length of the binary state vector was 460. The shallow network agents were trained for 200,000 episodes and the experiments were repeated for ten separate runs for each type of activation unit.

In the deep reinforcement learning setting, we used a deep network architecture consisting of two convolutional layers with 15 and 50 filters of size 5×5 using a stride of 1, a fully-connected layer with 250 units, and a linear output layer. Both convolutional layers were followed by max-pooling layers with pooling windows of size 3×3 using a stride of 2. We compared networks with SiLUs in both the convolutional and fully-connected layers (SiLU–SiLU) with networks with ReLUs in all hidden layers (ReLU–ReLU). Based on the high performance of the shallow networks with dSiLUs in the hidden layer, we also tested a deep network with dSiLUs in the last, fully-connected, hidden layer, combined with SiLUs in the convolutional layers (SiLU–dSiLU). The deep network agents were trained for 200,000 episodes and the experiments were repeated for five separate runs for each type of network.

We used the following reward function (proposed by Faußer and Schwenker (2013)):

$$r(\mathbf{s}) = e^{-(\text{number of holes in } \mathbf{s})/33}. \quad (14)$$

We set γ to 0.99, λ to 0.55, τ_0 to 0.5, and τ_k to 0.00025. We used a rough grid-like search to find appropriate values of the learning rate α and it was determined to be 0.001 for the four shallow network agents and 0.0001 for the three deep network agents.

¹ <http://barbados2011.rl-community.org/program/SzitaTalk.pdf>.

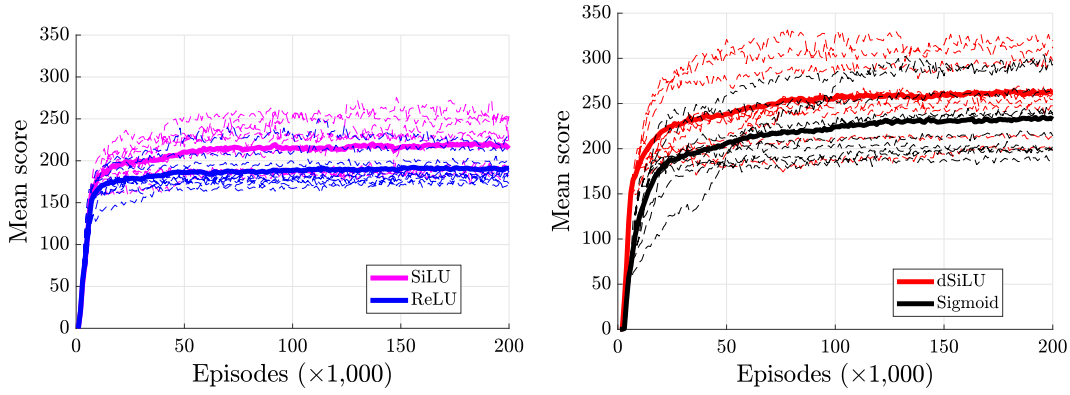


Fig. 2. Learning curves in stochastic SZ-Tetris for the four types of shallow neural network agents. The figure shows the average scores over ten separate runs (tick solid lines) and the scores of individual runs (thin dashed lines). The mean scores were computed over every 1000 episodes.

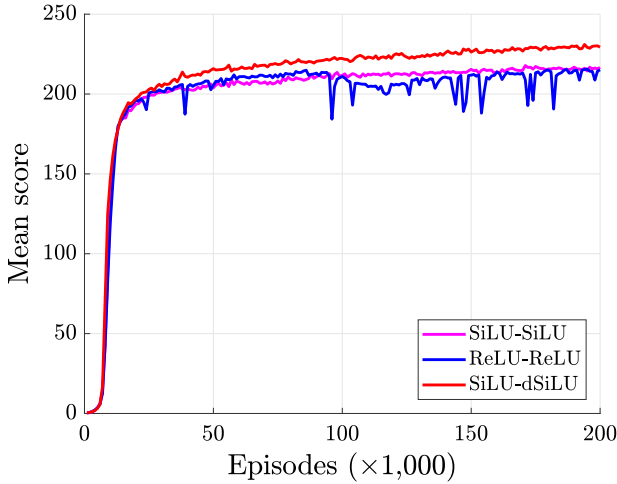


Fig. 3. Average Learning curves in stochastic SZ-Tetris for the three types of deep neural network agents. The figure shows the average scores over five separate runs, computed over every 1000 episodes.

Table 1

Average scores ($\pm$ standard deviations) achieved in stochastic SZ-Tetris, computed over the final 1000 episodes for all runs and the best single runs.

Network	Final average score	Final best score
Shallow networks		
SiLU	214 $\pm$ 74	253 $\pm$ 83
ReLU	191 $\pm$ 58	227 $\pm$ 76
dSiLU	263 $\pm$ 80	320 $\pm$ 87
Sigmoid	232 $\pm$ 75	293 $\pm$ 73
Deep networks		
SiLU-SiLU	217 $\pm$ 53	219 $\pm$ 54
ReLU-ReLU	215 $\pm$ 54	217 $\pm$ 52
SiLU-dSiLU	229 $\pm$ 55	235 $\pm$ 54

Fig. 2 shows the average learning curves as well as learning curves for the individual runs for the shallow networks, Fig. 3 shows the average learning curves for the deep networks, and the final results are summarized in Table 1. The results show significant differences ($p < 0.0001$) in final average score between all four shallow agents. The networks with bounded hidden units (dSiLU and sigmoid) outperformed the networks with unbounded units (SiLU and ReLU), the SiLU network outperformed the ReLU network, and the dSiLU network outperformed the sigmoid network. The final average score (best score) of 263 (320) points achieved by the dSiLU network agent is a new state-of-the-art score, improving the previous best performance by 43 (25) points

or 20% (8%). In the deep learning setting, the SiLU-dSiLU network significantly ($p < 0.0001$) outperformed the other two networks and the average final score of 229 points is better than the previous state-of-the-art of 220 points. There were no significant difference ($p = 0.32$) between the final performance of the SiLU-SiLU network and the ReLU-ReLU network.

3.2. 10×10 Tetris

The result achieved by the dSiLU network agent in stochastic SZ-Tetris is impressive, but we cannot compare the result with the methods that have achieved the highest performance levels in standard Tetris, because those methods have not been applied to stochastic SZ-Tetris. Furthermore, it is not feasible to apply our method to Tetris with a standard board height of 20, because of the prohibitively long learning time. The current state-of-the-art for a single run of an algorithm, achieved by the CBMPI algorithm (Gabillon, Ghavamzadeh, & Scherrer, 2013; Scherrer, Ghavamzadeh, Gabillon, Lesner, & Geist, 2015), is a mean score of 51 million cleared lines. However, for the best methods applied to Tetris, there are reported results for a smaller, 10×10 , Tetris board, and in this case the learning time for our method is long, but not prohibitively so.

10×10 Tetris is played with the standard seven tetrominos and the numbers of actions are 9 for the block-shaped tetromino, 17 for the S-, Z-, and stick-shaped tetrominos, and 34 for the J-, L- and T-shaped tetrominos. In each time step, the agent gets a score equal to the number of completed rows, with a maximum of +4 points that can only be achieved by the stick-shaped tetromino.

We trained a shallow neural network agent with dSiLU units in the hidden layer. To handle the more complex learning task, we increased the number of hidden units to 250 and the number of episodes to 400,000. We repeated the experiment for five separate runs. We used the same 20 state features as in the SZ-Tetris experiment, but the length of the binary state vector was reduced to 260 due to the smaller board size. The reward function was changed as follows for the same reason:

$$r(s) = e^{-(\text{number of holes in } s)/(33/2)}. \quad (15)$$

We used the same values of the meta-parameters as in the stochastic SZ-Tetris experiment.

The average learning curve as well as learning curves for the five separate runs in 10×10 Tetris are shown in Fig. 4. The dSiLU network agent reached an average score of 4900 points over the final 10,000 episodes and the five separate runs, which is a new state-of-the-art in 10×10 Tetris. The previous best average scores are 4200 points achieved by the CBMPI algorithm, 3400 points achieved by the DPI algorithm, and 3000 points achieved by the

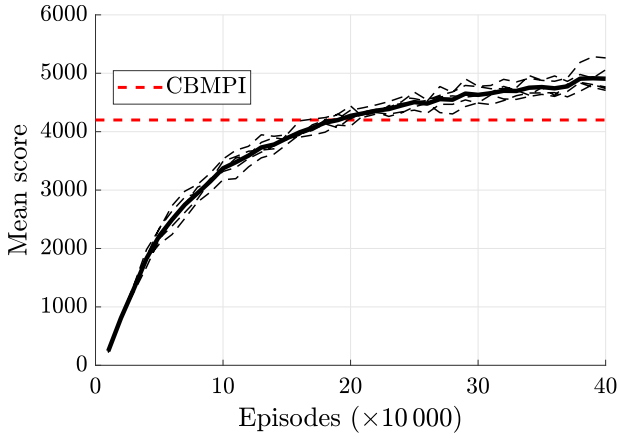


Fig. 4. Learning curves for a dSiLU network agent with 250 hidden nodes in 10×10 Tetris. The figure shows the average score over five separate runs (thick solid lines) and the scores of individual runs (thin dashed lines). The red dashed line shows the previous best average score of 4200 points achieved by the CBMPI algorithm.

CE method (Gabillon et al., 2013). The best individual run achieved a final mean score of 5300 points, which is also a new state-of-the-art, improving on the score of 5000 points achieved by the CBMPI algorithm.

It is particularly impressive that the dSiLU network agent achieved its result using features similar to the original Bertsekas features. Using only the Bertsekas features, the CBMPI algorithm, the DPI algorithm, and the CE method could only achieve average scores of about 500 points (Gabillon et al., 2013). The CE method has achieved its best score by combining the Bertsekas features, the Dellacherie features (Fahey, 2003), and three original features (Thiery & Scherrer, 2009). The CBMPI algorithm achieved its best score using the same features as the CE method, except for using five original RBF height features instead of the Bertsekas features.

3.3. Atari 2600 games

To further evaluate the use of value-based on-policy reinforcement learning with eligibility traces and softmax action selection in high-dimensional state space domains we applied Sarsa(λ) with a deep convolution neural network function approximator in the Atari 2600 domain using the Arcade Learning Environment (Bellemare, Naddaf, Veness, & Bowling, 2013). Based on the results for the deep networks in SZ-Tetris, we used a SiLU–dSiLU network with SiLU units in the convolutional layers and dSiLU units in the fully-connected layer. To limit the number of games and prevent a biased selection of the games, we selected the 12 games played by DQN (Mnih et al., 2015) that begin with the letters ‘A’ and ‘B’: Alien, Amidar, Assault, Asterix, Asteroids, Atlantis, Bank Heist, Battle Zone, Beam Rider, Bowling, Boxing, and Breakout.

We used a similar experimental setup as Mnih et al. (2015). We pre-processed the raw 210×160 Atari 2600 RGB frames by extracting the luminance channel, taking the maximum pixel values over consecutive frames to prevent flickering, and then downsampling the grayscale images to 105×80 . For computational reasons, we used a smaller network architecture. Instead of three convolutional layers, we used two with half the number of filters, each followed by a max-pooling layer. The input to the network was a $105 \times 80 \times 2$ image consisting of the current and the fourth previous pre-processed frame. As we used frame skipping where actions were selected every fourth frame and repeated for the next four frames, we only needed to apply pre-processing to every fourth frame. The first convolutional layer had 16 filters of

size 8×8 with a stride of 4. The second convolutional layer had 32 filters of size 4×4 with a stride of 2. The max-pooling layers had pooling windows of size 3×3 with a stride of 2. The convolutional layers were followed by a fully-connected hidden layer with 512 dSiLU units and a fully-connected linear output layer with 4 to 18 output (or action-value) units, depending on the number of valid actions in the considered game. We selected meta-parameters by a preliminary search in the Alien, Amidar and Assault games and used the same values for all 12 games: α : 0.001, γ : 0.99, λ : 0.8, τ_0 : 0.5, and τ_k : 0.0005. As in Mnih et al. (2015), we clipped the rewards to be between -1 and $+1$, but we did not clip the values of the TD-errors.

In each of the 12 Atari games, we trained a SiLU–dSiLU agent for 200,000 episodes and the experiments were repeated for two separate runs. An episode started with up to 30 ‘do nothing’ actions (*no-op condition*) and it was played until the end of the game or for a maximum of 18,000 frames (i.e., 5 min). We evaluated the agents by computing the mean scores over every 100 consecutive episodes in each run, which we call the *mScore*. Figs. 5 and 6 show the average learning curves, as well as the learning curves for the two separate runs, in the 12 Atari 2600 games. Table 2 summarizes our results using two metrics:

- **Final score:** the average score computed over the final mScores.
- **Best score:** the average score computed over the maximum mScores.

We included the Best score to be able to compare our results with those achieved by DQN (single run scores, where the mScores were computed over 30 episodes; Mnih et al., 2015), the Gorilla implementation of DQN (average scores over 5 runs, where the mScores were computed over 30 episodes; Nair et al., 2015), and double DQN (singles run scores, where the mScores were computed over 100 episodes; van Hasselt et al., 2015). The last two rows of the table shows summary statistics over the 12 games, which were obtained by computing the mean and the median of the DQN normalized scores:

$$\text{Score}_{\text{DQN_normalized}} = \frac{\text{Score}_{\text{agent}} - \text{Score}_{\text{random}}}{\text{Score}_{\text{DQN}} - \text{Score}_{\text{random}}}.$$

Here, $\text{Score}_{\text{random}}$ is the score achieved by a random agent in Mnih et al. (2015).

The results clearly show that our SiLU–dSiLU agent outperformed the other agents, improving the mean (median) DQN normalized Best score from 127% (105%) achieved by double DQN to 332% (125%). The SiLU–dSiLU agents achieved the highest Best score in 6 out of the 12 games and only performed much worse than the other 3 agents in one game, Breakout, where the learning never took off during the 200,000 episodes of training (see Fig. 6). The performance was especially impressive in the Asterix (Best score of 100,322) and Asteroids (Best score of 10,614) games, which improved the Best scores achieved by the second-best agent by 562% and 552%, respectively.

4. Analysis

4.1. Value estimation

First, we investigate the ability of TD(λ) and Sarsa(λ) to accurately estimate discounted returns:

$$R_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}.$$

Here T is the length of an episode. The reason for doing this is that van Hasselt et al. (2015) showed that the double DQN

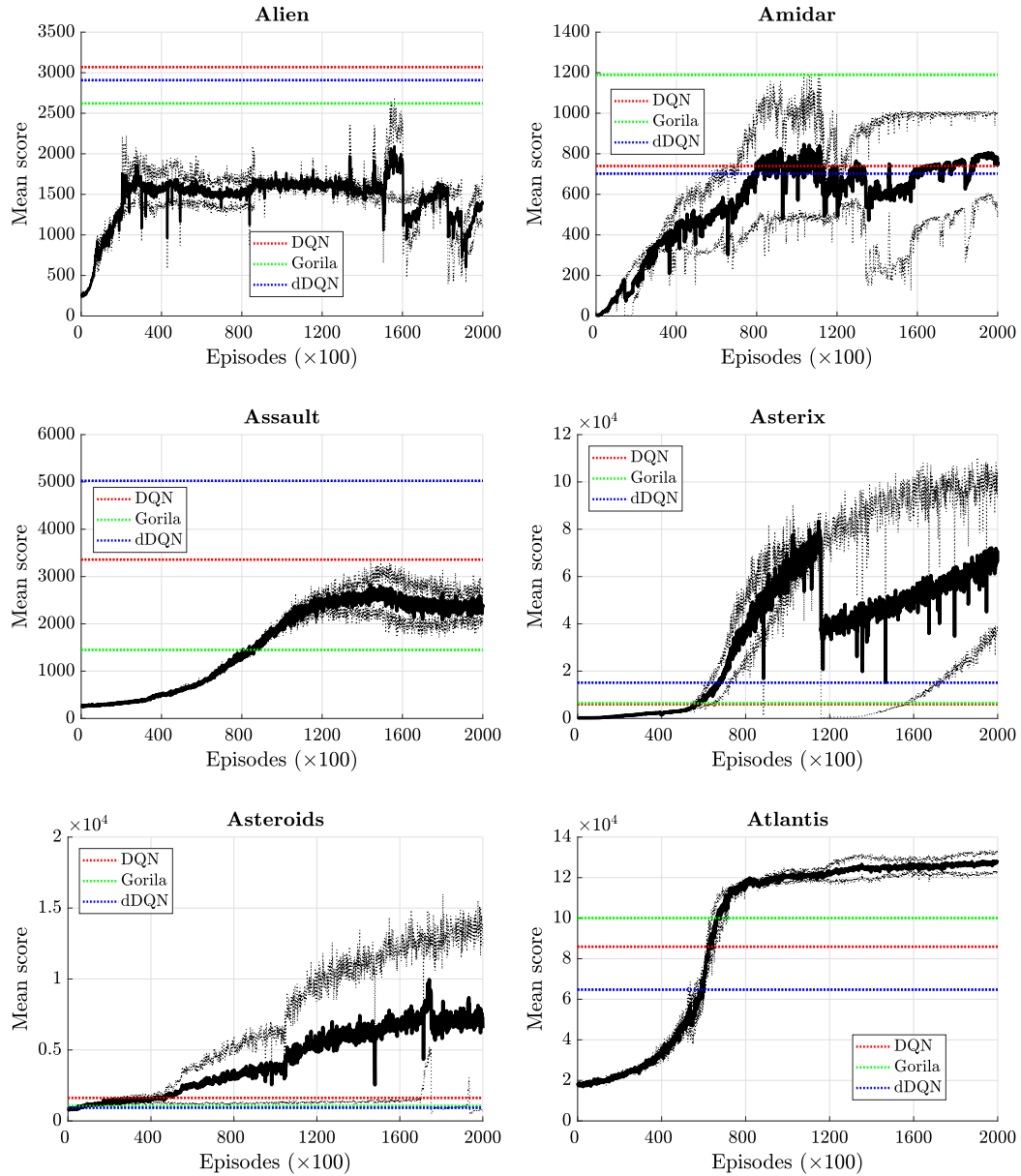


Fig. 5. Average learning curves (solid lines) over two separate runs (dashed lines) for the SiLU-dSiLU agents in the 6 Atari games that begin with the letter ‘A’. The dotted lines show the reported results for DQN (red), the Gorila implementation of DQN (green), and double DQN (blue). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 2

The Best and Final scores achieved by our SiLU-dSiLU agents in 12 Atari 2600 games, and the reported Best scores achieved by DQN, the Gorila implementation of DQN, and double DQN (dDQN) in the no-op condition with 5 min of evaluation time.

Game	DQN	Gorila	dDQN	SiLU-dSiLU	
	Best	Best	Best	Final	Best
Alien	3,069	2,621	2,907	1,370	2,246
Amidar	740	1,190	702	762	904
Assault	3,359	1,450	5,023	2,415	2,944
Asterix	6,012	6,433	15,150	70,942	100,322
Asteroids	1,629	1,048	931	6,537	10,614
Atlantis	85,950	100,069	64,758	127,651	128,983
Bank Heist	430	609	728	5	770
Battle Zone	26,300	25,267	25,730	22,930	29,115
Beam Rider	6,846	3,303	7,654	1,829	2,176
Bowling	42	54	71	67	75
Boxing	72	95	82	36	92
Breakout	401	402	375	25	55
Mean (DQN Normalized)	100%	102%	127%	218%	332%
Median (DQN Normalized)	100%	104%	105%	78%	125%

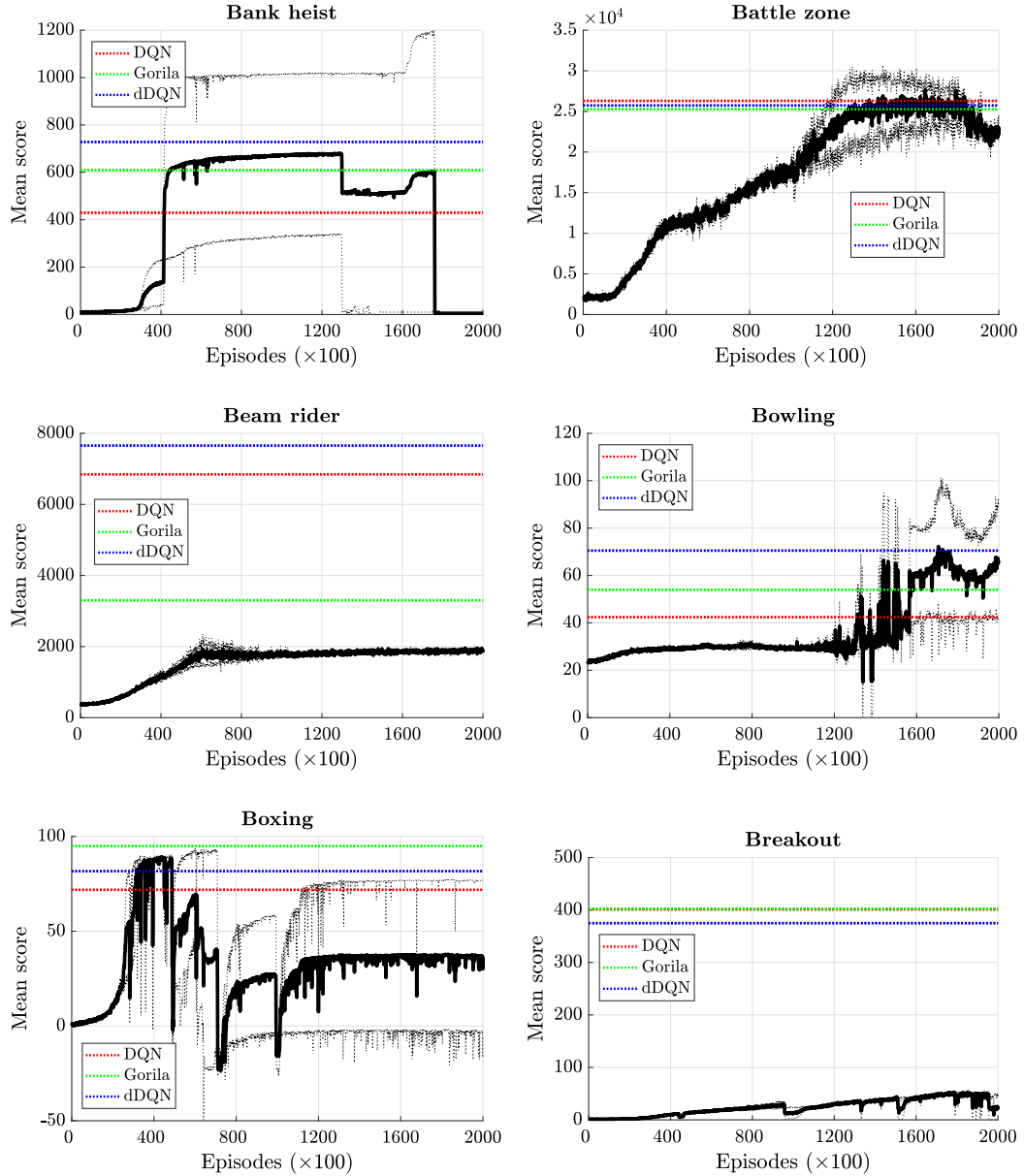


Fig. 6. Average learning curves (solid lines) over two separate runs (dashed lines) for the SiLU-dSiLU agents in the 6 Atari games that begin with the letter 'B'. The dotted lines show the reported results for DQN (red), the Gorila implementation of DQN (green), and double DQN (blue). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

algorithm improved the performance of DQN in Atari 2600 games by reducing the overestimation of the action values. It is known (Thrun & Schwartz, 1993; van Hasselt, 2010) that Q-learning based algorithms, such as DQN, can overestimate action values due to the max operator, which is used in the computation of the learning targets. TD(λ) and Sarsa(λ) do not use the max operator to compute the learning targets and they should therefore not suffer from this problem.

Fig. 7 shows that for SZ-Tetris episodes of average (or expected) length, the best dSiLU network agent at the end of the learning ($\tau = 0.0098$) learned good estimates of the discounted returns, both along the episodes (left panel) and as measured by the normalized sum of differences between $V(s_t)$ and R_t (right panel):

$$\frac{1}{T} \sum_{t=1}^T (V(s_t) - R_t).$$

The linear fit of the normalized sum of differences data for 1000 episodes gives a small underestimation (-0.43) for an episode of

average length (866 time steps). The $V(s_t)$ -values overestimated the discounted returns for short episodes and underestimated the discounted returns for long episodes (especially in the middle part of the episodes), which is accurate since the episodes ended earlier and later, respectively, than were expected.

Fig. 8 shows typical examples of learned action values and discounted returns along episodes in Asterix (score of 108,500) and Asteroids (score of 22,500), when the best SiLU-dSiLU agents at the end of the learning ($\tau = 0.00495$) successfully played for the full 18,000 frames (i.e., 4500 time steps since the agents acted every fourth frame). In both games, with the exception of a few smaller parts, the learned action values matched the discounted returns very well along the whole episodes. The normalized sums of differences (absolute differences) were 0.59 (1.05) in the Asterix episode and -0.23 (1.28) in the Asteroids episode. In both games, the agents overestimated action values at the end of the episodes. However, this is an artifact of that an episode ended after a maximum of 4500 time steps, which the agents could not predict.

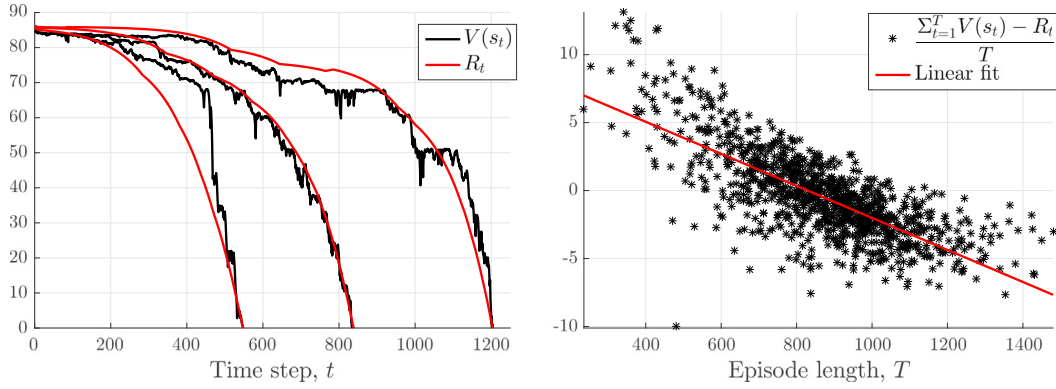


Fig. 7. The left panel shows learned $V(s_t)$ -values and R_t -values, for examples of short, medium-long, and long episodes in SZ-Tetris. The right panel shows the normalized sum of differences between $V(s_t)$ and R_t for 1000 episodes and the best linear fit of the data ($-0.012T + 9.8$).

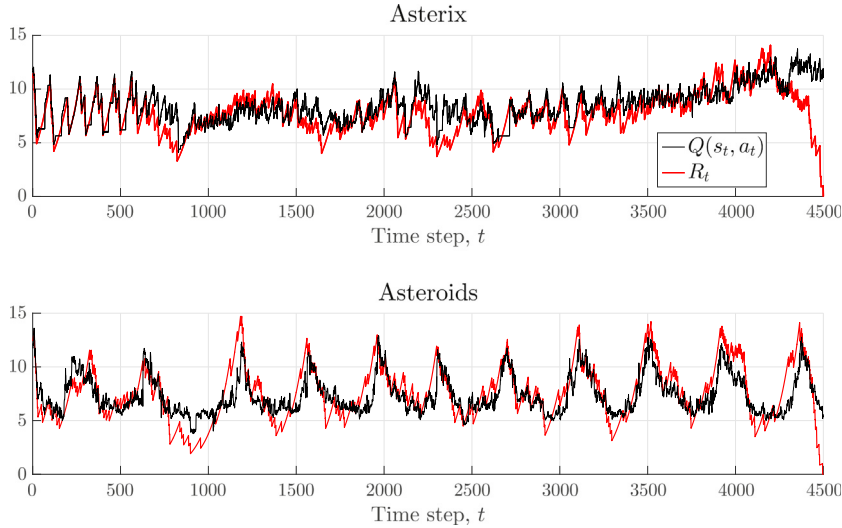


Fig. 8. Learned action values, $Q(s_t, a_t)$, and discounted returns, R_t , for the best SiLU-dSiLU agents in Asterix and Asteroids.

Videos of the corresponding learned behaviors in Asterix and Asteroids can be found at http://www.cns.atr.jp/~elfwing/videos/asterix_deep_SiLU.mov and http://www.cns.atr.jp/~elfwing/videos/asteroids_deep_SiLU.mov.

4.2. Action selection

Second, we investigate the importance of softmax action selection in the games where our proposed agents performed particularly well. Almost all deep reinforcement learning algorithms that have been used in the Atari 2600 domain have used ϵ -greedy action selection (one exception is the asynchronous advantage actor-critic method, A3C, which used softmax output units for the actor Mnih et al., 2016). One drawback of ϵ -greedy selection is that it selects all actions with equal probability when exploring, which can lead to poor learning outcomes in tasks where the worst actions have very bad consequences. This is clearly the case in both Tetris games and in the Asterix and Asteroids games. In each state in Tetris, many, and often most, actions will create holes, which are difficult (especially in SZ-Tetris) to remove. In the Asterix game, random exploratory actions can kill Asterix if executed when Cacophonix's deadly lyres are passing. In the Asteroids game, one of the actions sends the spaceship into hyperspace and makes it reappear in a random location, which has the risk of the spaceship self-destructing or of destroying it by appearing on top of an asteroid.

We compared softmax action selection, where τ was set to the annealed values after 10,000 episodes (τ_{10k}), 50,000 episodes (τ_{50k}), and 200,000 episodes (τ_{200k} , i.e., the τ -values at the end of learning), and ϵ -greedy action selection, where ϵ was set to 0, 0.001, 0.01, and 0.05, for the best dSiLU network agent in SZ-Tetris and the best SiLU-dSiLU agents in the Asterix and Asteroids games. The results (see Table 3) clearly show that ϵ -greedy action selection with ϵ set to 0.05, as used for evaluation by DQN, is not suitable for these games. The scores were only 4% to 10% of the scores for softmax selection using τ_{200k} . The negative effects of random exploration were largest in Asteroid and SZ-Tetris. Even when ϵ was set as low as 0.001 and the agent performed only 2.1 non-greedy actions per episode in Asteroids and 0.6 in SZ-Tetris, the mean scores were reduced by 26% and 22%, respectively, compared with $\epsilon = 0$. In contrast, even if the numbers of non-greedy actions in Asteroids and SZ-Tetris were increased by about an order of magnitude when using τ_{10k} compared to when using τ_{200k} , the scores were only reduced by 10% and 7%, respectively.

5. Conclusions

In this study, we proposed SiLU and dSiLU as activation functions for neural network function approximation in reinforcement learning. We demonstrated in stochastic SZ-Tetris that SiLUs significantly outperformed ReLUs, and that dSiLUs significantly outperformed sigmoid units. The best agent, the dSiLU network

Table 3

Mean scores and average numbers of non-greedy actions for softmax action selection and ϵ -greedy action selection.

Game	Selection	Mean score	Non-greedy actions
SZ-Tetris	$\tau_{200k} = 0.0098$	326	28.7
	$\tau_{50k} = 0.0370$	318	93.0
	$\tau_{10k} = 0.1429$	302	191.7
	$\epsilon = 0$	332	0
	$\epsilon = 0.001$	260	0.6
	$\epsilon = 0.01$	71	2.0
	$\epsilon = 0.05$	14	3.2
Asterix	$\tau_{200k} = 0.00495$	104,299	47.6
	$\tau_{50k} = 0.0192$	97,365	178.5
	$\tau_{10k} = 0.0833$	71,505	576.9
	$\epsilon = 0$	102,890	0
	$\epsilon = 0.001$	98,264	3.6
	$\epsilon = 0.01$	66,113	30.0
	$\epsilon = 0.05$	7,152	56.8
Asteroids	$\tau_{200k} = 0.00495$	15,833	31.3
	$\tau_{50k} = 0.0192$	15,421	92.7
	$\tau_{10k} = 0.0833$	14,219	355.2
	$\epsilon = 0$	15,091	0
	$\epsilon = 0.001$	11,105	2.1
	$\epsilon = 0.01$	3,536	11.7
	$\epsilon = 0.05$	1,521	47.3

agent, achieved a new state-of-the-art in stochastic SZ-Tetris and in 10×10 Tetris. In the Atari 2600 domain, a deep Sarsa(λ) agent with SiLUs in the convolutional layers and dSiLUs in the fully-connected hidden layer outperformed DQN and double DQN, as measured by mean and median DQN normalized scores.

An additional purpose of this study was to demonstrate that a more traditional approach of using on-policy learning with eligibility traces and softmax selection (i.e., basically a “textbook” version of a reinforcement learning agent but with non-linear neural network function approximators) can be competitive with the approach used by DQN. This means that there is a lot of room for improvements, by, e.g., using, as DQN, a separate target network, but also by using more recent advances such as the dueling architecture (Wang et al., 2016) for more accurate estimates of the action values and asynchronous learning by multiple agents in parallel (Mnih et al., 2016).

Acknowledgments

This work was supported by the project commissioned by the New Energy and Industrial Technology Development Organization (NEDO), MEXT KAKENHI grants 16H06563 and 17H06042, and Okinawa Institute of Science and Technology Graduate University research support to KD.

References

- Bellemare, M. G., Naddaf, Y., Veness, J., & Bowling, M. (2013). The Arcade learning environment: an evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47, 253–279.
- Bertsekas, D. P., & Ioffe, S. (1996). Temporal differences based policy iteration and applications in neuro-dynamic programming. Technical Report LIDS-P-2349, MIT.
- Burgiel, H. (1997). How to lose at Tetris. *Mathematical Gazette*, 81, 194–200.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *CVPR09*.
- Elfving, S., Uchibe, E., & Doya, K. (2015). Expected energy-based restricted Boltzmann machine for classification. *Neural Networks*, 64(3), 29–38.
- Elfving, S., Uchibe, E., & Doya, K. (2016). From free energy to expected energy: Improving energy-based value function approximation in reinforcement learning. *Neural Networks*, 84, 17–27.
- Elfving, S., Uchibe, E., & Doya, K. (2017). Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *arXiv:1702.03118* [cs.LG].

- Fahey, C. (2003). Tetris AI, computer plays tetris. colinfahey.com/tetris/tetris.html [Online; (accessed 22.02.17)].
- Faußer, S., & Schwenker, F. (2013). Neural network ensembles in reinforcement learning. *Neural Processing Letters*, 1–15.
- Freund, Y., & Haussler, D. (1992). Unsupervised learning of distributions on binary vectors using two layer networks. In J. E. Moody, S. J. Hanson, & R. P. Lippmann (Eds.), *Proceedings of advances in neural information processing systems*. Morgan Kaufmann.
- Gabillon, V., Ghavamzadeh, M., & Scherrer, B. (2013). Approximate dynamic programming finally performs well in the game of tetris. In *Proceedings of advances in neural information processing systems* (pp. 1754–1762).
- Hahnloser, R. H. R., Sarpeshkha, R., Mahowald, M. A., Douglas, R. J., & Seung, H. S. (2000). Digital selection and analogue amplification coexist in a cortex-inspired silicon circuit. *Nature*, 405, 947–951.
- Hinton, G. E. (2002). Training products of experts by minimizing contrastive divergence. *Neural Computation*, 12(8), 1771–1800.
- Jaskowski, W., Szubert, M. G., Liskowski, P., & Krawiec, K. (2015). High-dimensional function approximation for knowledge-free reinforcement learning: a case study in SZ-Tetris. In *Proceedings of the genetic and evolutionary computation conference* (pp. 567–573).
- Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Tech. rep., University of Toronto.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., & Harley, T. et al., (2016). Asynchronous methods for deep reinforcement learning. In *Proceedings of the international conference on machine learning* (pp. 1928–1937).
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Nair, A., Srinivasan, P., Blackwell, S., Alceick, C., Fearon, R., & Maria, A. D. et al., (2015). Massively parallel methods for deep reinforcement learning. *arXiv:1507.04296* [cs.LG].
- Ramachandran, P., Zoph, B., & Le, Q. V. (2017). Searching for activation functions. *arXiv:1710.05941* [cs.NE].
- Rummery, G. A., & Niranjan, M. (1994). On-line Q-learning using connectionist systems. Tech. Rep. CUED/F-INFENG/TR 166, Cambridge University Engineering Department.
- Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized experience replay. In *International conference on learning representations*, Puerto Rico.
- Scherrer, B., Ghavamzadeh, M., Gabillon, V., Lesner, B., & Geist, M. (2015). Approximate modified policy iteration and its application to the game of tetris. *Journal of Machine Learning Research (JMLR)*, 16, 1629–1676.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529, 484–503.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., & Guez, A. et al., (2017). Mastering the game of go without human knowledge. Vol. 550, (pp. 354–359).
- Smolensky, P. (1986). Information processing in dynamical systems: Foundations of harmony theory. In D. E. Rumelhart, & J. L. McClelland (Eds.), *Parallel distributed processing: explorations in the microstructure of cognition. Volume 1: Foundations*. MIT Press.
- Sutton, R. S. (1988). Learning to predict by the method of temporal differences. *Machine Learning*, 3, 9–44.
- Sutton, R. S. (1996). Generalization in reinforcement learning: successful examples using sparse coarse coding. In *Proceedings of advances in neural information processing systems* (pp. 1038–1044). MIT Press.
- Sutton, R. S., & Barto, A. (1998). *Reinforcement learning: An introduction*. MIT Press.
- Szita, I., & Szepesvári, C. (2010). SZ-Tetris as a benchmark for studying key problems of reinforcement learning. In *ICML 2010 workshop on machine learning and games*.
- Tesauro, G. (1994). TD-Gammon, a self-teaching backgammon program, achieves master-level play. *Neural Computation*, 6(2), 215–219.
- Thiery, C., & Scherrer, B. (2009). Improvements on learning tetris with cross entropy. *International Computer Games Association Journal*, 32.
- Thrun, S., & Schwartz, A. (1993). Issues in using function approximation for reinforcement learning. In *Proceedings of the 1993 connectionist models summer school* (pp. 255–263).
- van Hasselt, H. (2010). Double q-learning. In *Proceedings of advances in neural information processing systems* (pp. 2613–2621).
- van Hasselt, H., Guez, A., & Silver, D. (2015). Deep Reinforcement learning with double q-learning. *arXiv:1509.06461* [cs.LG].
- Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In *Proceedings of the international conference on machine learning* (pp. 1995–2003).